
1. A "Project Glossary" Document (critical for AI consistency)

The AI will forget your table/column names between units. Create a `GLOSSARY.md` with:

Database

- users: id, email, password_hash, role (admin/user), created_at
- products: id, name, description, price, category_id, image_path
- categories: id, name, slug
- files: id, original_name, stored_name, mime_type, size, user_id, created_at

Foreign Keys

- products.category_id → categories.id (ON DELETE SET NULL)
- files.user_id → users.id (ON DELETE CASCADE)

URL Convention

- Route format: index.php?route=controller/action/id
- Example: index.php?route=products/show/5

Why: When you ask for "ProductController," the AI won't guess your column names. Paste this glossary into every complex prompt.

2. A `.env.example` + Config Loader Strategy

The PDFs mention `config/db_config.php` but don't specify how it's loaded. Create this before starting:

`config/config.php` (loaded by `index.php`):

```
<?php
return [
    'db' => [
        'host' => 'localhost',
        'dbname' => 'your_db',
        'user' => 'root',
        'password' => '',
        'charset' => 'utf8mb4'
    ],
    'upload' => [
        'max_size' => 2 * 1024 * 1024,
        'allowed_types' => ['jpg', 'png', 'pdf'],
    ]
];
```

```

        'upload_dir' => __DIR__ . '/../public/uploads/'
    ],
    'app' => [
        'url' => 'http://localhost:8000', // for asset generation
        'session_name' => 'myapp_session'
    ]
];

```

Why: The Singleton Database class needs credentials. Hardcoding fails in production. This gives you one source of truth.

3. A Simple Autoloader Decision (before Unit 1)

The PDFs suggest manual `require_once` (page 5 of doc 3 says "I'm not using an autoloader"). This will become **unmaintainable** after 10+ classes.

Recommendation: Decide now between:

- **Option A (simpler for vibe coding):** Use Composer's PSR-4 autoloader (install once, forget). Add `composer.json`:

```

{
    "autoload": {
        "psr-4": {
            "App\\": ""
        }
    }
}

```

- **Option B (pure manual):** Create an `init.php` that includes all classes. But you'll fight path issues.

My advice: Use Composer. It adds 2 minutes of setup but saves hours of debugging "class not found."

4. A "Test Each Unit" Template Script

The PDFs mention testing each unit but don't give a template. Create `tests/test_unit.php`:

```

<?php
error_reporting(E_ALL);
ini_set('display_errors', 1);

```

```

require_once __DIR__ . '/../config/config.php';
require_once __DIR__ . '/../utils/Database.php';

// Test Database singleton
$db = Database::getInstance();
echo "PDO connected: " . get_class($db) . "\n";

// Test UserDao (after Unit 3)
$userDao = new UserDao($db);
$users = $userDao->findAll();
echo "Found " . count($users) . " users\n";

```

Run after each unit before moving on. This catches integration failures early.

5. A "Prompt Library" with Context Injection

The PDF prompts are good but lack **context**. For each unit, create a prompt template that pastes previous work. Example for Unit 3 (DAO):

```

You are writing a DAO for a 'products' table.
Here is the exact table schema from Unit 0:
[PASTE database.sql]

Here is the Product entity class from Unit 2:
[PASTE Product.php]

Here is the Database singleton from Unit 1:
[PASTE Database.php]

Write ProductDao.php with find($id), findAll(), save($product), delete($id).
Use prepared statements. Return only the code.

```

Why: Without this, the AI will guess column names or invent methods. Context = reliability.

6. A "Rollback & Compare" Strategy for AI Code

When the AI gives code that doesn't work, you need a systematic way to debug. Prepare:

- **Git commit before each unit** (so you can `git diff` to see what changed)
- A `debug.log` where you paste AI responses and actual outputs
- A **"minimal reproduction" pattern** – when something fails, create a `test_fail.php` with only 10 lines that isolates the issue

Example minimal reproduction for login failure:

```
<?php
$password = 'test123';
$hash = '$2y$10$...'; // from database
var_dump(password_verify($password, $hash)); // false? Then issue is hash
mismatch
```

What You Already Have Right

- ✓ Unit dependency map (doc 2, page 6) – excellent
- ✓ Error-fix prompts (doc 3, pages 3-5) – these will save you
- ✓ AI report table (doc 1, page 2) – required for submission
- ✓ Realistic scope – 3 tables, CRUD, upload, search – perfectly sized

Final Recommendation – Before You Start

1. **Create the 6 documents above** (1 hour of prep)
2. **Run Unit 0 (SQL) and manually import it** into MySQL to verify no syntax errors
3. **Test Unit 1 (Database singleton) in isolation** – paste the code into `test_db.php` and run it
4. **Decide on autoloader** (Composer or manual) – stick with it
5. **Set up your Git repo** with initial commit of these preparation docs

Then start Unit 0 → Unit 1 → etc., testing each before moving on.

You're better prepared than 90% of developers who vibe code. The missing pieces are **context preservation** (glossary, config, autoloader) and **test automation**. Add those, and you'll finish without getting stuck.

Want me to generate the **glossary + config loader + test script** files for you to drop into your project?